

TÀI LIỆU THIẾT KẾ PHẦN MỀM (SDD)

Nhóm dự án VTIT

Tháng 6 năm 2026

0 Mục lục

1	Kiến trúc hệ thống (System Architecture)	1
1.1	Tổng quan kiến trúc (Architecture Overview)	1
1.2	Công nghệ áp dụng (Tech Stack)	1
2	Thiết kế Cơ sở dữ liệu (Database Design)	2
2.1	Sơ đồ Thực thể - Mối quan hệ (ERD Overview)	2
2.2	Thiết kế JSONB linh hoạt cho CV	2
3	Thiết kế Module (Component Design)	3
3.1	Các Module Frontend	3
3.2	Các Service Backend	3
4	Thiết kế Giao diện Lập trình (API Design)	4
4.1	Danh sách API quan trọng	4
5	Thiết kế Xử lý Bất đồng bộ (Async Processing Design)	5
5.1	1. Gửi Email thông báo (Notification Worker)	5
5.2	2. Nhắc nhở định kỳ (Cronjob)	5
6	Kiến trúc Triển khai (Deployment Architecture)	6
6.1	Môi trường đóng gói (Docker)	6

1 Kiến trúc hệ thống (System Architecture)

1.1 Tổng quan kiến trúc (Architecture Overview)

Dự án UmiCV sử dụng kiến trúc Client-Server với mô hình Monolithic (hoặc Modular Monolith) ở giai đoạn Release 1, đảm bảo dễ triển khai và bảo trì. Các thành phần chính bao gồm:

- **Client (Frontend):** Ứng dụng Web Single Page Application (SPA), tương tác với Backend qua RESTful API.
- **API Server (Backend):** Xử lý nghiệp vụ chính, bảo mật, xác thực và phân quyền.
- **Database:** PostgreSQL lưu trữ dữ liệu người dùng, cấu trúc tổ chức và nội dung CV (sử dụng tính năng JSONB cho các schema linh động).
- **Message Broker:** Redis kết hợp với BullMQ (hoặc Celery/RabbitMQ tương đương) dùng để xử lý các tác vụ bất đồng bộ như gửi Email thông báo.
- **Background Worker:** Các tiến trình Worker chạy độc lập để tiêu thụ (consume) các message từ Broker và xử lý cronjob (nhắc nhở định kỳ).

1.2 Công nghệ áp dụng (Tech Stack)

- **Frontend:** React, Vite, TailwindCSS, shadcn/ui, TypeScript.
- **Backend:** Node.js, ExpressJS, TypeScript cung cấp API chuẩn REST.
- **Database:** PostgreSQL.
- **Caching & Queue:** Redis.
- **Deployment:** Docker & Docker Compose.

2 Thiết kế Cơ sở dữ liệu (Database Design)

2.1 Sơ đồ Thực thể - Mối quan hệ (ERD Overview)

Hệ thống bao gồm các bảng (Entities) cốt lõi sau:

1. **users**: Lưu trữ thông tin tài khoản, mật khẩu băm, quyền hạn (Role).
2. **departments**: Quản lý danh mục phòng ban, dự án.
3. **cv_profiles**: Lưu trữ thông tin cơ bản của CV, liên kết với **users**.
4. **cv_sections** (hoặc JSONB trong **cv_profiles**): Lưu trữ cấu trúc động của CV.
5. **batch_requests**: Yêu cầu cập nhật CV hàng loạt từ HR.
6. **approval_logs**: Ghi nhận lịch sử duyệt, lý do từ chối (Audit Logs).

2.2 Thiết kế JSONB linh hoạt cho CV

Để đáp ứng yêu cầu linh hoạt của Trình tạo CV (CV Builder) theo FR-04, nội dung chi tiết của các mục (sections) như Học vấn, Kinh nghiệm, Kỹ năng được lưu dưới dạng JSONB trong PostgreSQL.

Mẫu Schema JSONB dự kiến:

```
{
  "sections": [
    {
      "id": "sec-1",
      "type": "education",
      "title": "Học vấn",
      "order": 1,
      "data": [
        {
          "school": "Đại học Bách Khoa",
          "degree": "Kỹ sư CNTT",
          "start_date": "2018-09",
          "end_date": "2023-06"
        }
      ]
    },
    {
      "id": "sec-2",
      "type": "experience",
      "title": "Kinh nghiệm làm việc",
      "order": 2,
      "data": []
    }
  ]
}
```

3 Thiết kế Module (Component Design)

3.1 Các Module Frontend

- **Auth Module:** Quản lý đăng nhập, quản lý token (lưu Refresh token qua HTTP-only cookie), interceptors để đính kèm Access Token.
- **CV Builder Component:** Giao diện kéo thả (Drag & Drop), quản lý state toàn cục (Redux/Zustand), xử lý render nội dung động.
- **Diff Viewer Component:** Nhận 2 phiên bản JSON của CV, so sánh và hiển thị highlight (xanh/đỏ) cho Tech Lead/HR duyệt.
- **Dashboard & Report:** Vẽ biểu đồ, hiển thị danh sách công việc (To-do), tích hợp tải file Excel.

3.2 Các Service Backend

- **Auth Service:** Xử lý hash mật khẩu, sinh JWT, middleware kiểm tra quyền RBAC.
- **CV Service:** CRUD thao tác trên DB, quản lý Version (tạo bản ghi lịch sử khi Publish), quản lý Draft Space.
- **Workflow Service:** Kiểm soát luồng trạng thái (Nháp -> Chờ duyệt -> Đã cập nhật), ghi log lý do Reject.
- **Batch Request Service:** Khởi tạo yêu cầu, phân phối jobs vào Message Queue.

4 Thiết kế Giao diện Lập trình (API Design)

Các API tuân thủ tiêu chuẩn RESTful, trả về định dạng JSON.

4.1 Danh sách API quan trọng

Method	Endpoint	Mô tả
POST	/api/auth/login	Đăng nhập, trả về Access Token, set HTTP-only cookie Refresh Token.
POST	/api/auth/refresh	Cấp lại Access Token mới.
GET	/api/cvs/draft	Lấy bản nháp CV của user đang đăng nhập.
PUT	/api/cvs/draft	Lưu bản nháp CV (Upsert).
POST	/api/cvs/draft/submit	Gửi duyệt bản nháp (chuyển trạng thái Chờ duyệt).
POST	/api/cvs/{id}/approve	Duyệt CV theo cấp (Tech Lead / HR).
POST	/api/cvs/{id}/reject	Từ chối CV, yêu cầu reason trong body.
GET	/api/cvs/{id}/diff	Lấy dữ liệu chênh lệch giữa bản nháp và bản chính thức.
POST	/api/batch-requests	Tạo yêu cầu cập nhật CV hàng loạt (HR).

5 Thiết kế Xử lý Bất đồng bộ (Async Processing Design)

Hệ thống sử dụng cơ chế Background Worker cho hai nghiệp vụ chính:

5.1 1. Gửi Email thông báo (Notification Worker)

- **Producer:** Batch Request Service hoặc Workflow Service (khi tạo request mới, hoặc khi CV bị Reject).
- **Queue:** Một hàng đợi trên Redis (ví dụ: `email_queue`).
- **Consumer:** Worker lắng nghe `email_queue`, tiến hành kết nối tới SMTP Server để gửi mail.
- **Cơ chế:** Có retry logic nếu SMTP server tạm thời lỗi. Đảm bảo SLA gửi email dưới 5 phút.

5.2 2. Nhắc nhở định kỳ (Cronjob)

- **Lịch trình (Schedule):** Cấu hình chạy hằng ngày vào 8:00 AM (`0 8 * * *`).
- **Nghiệp vụ:** Quét bảng `batch_requests` và danh sách users có trạng thái CV Chưa cập nhật và gần đến `deadline`.
- **Hành động:** Đẩy các jobs gửi email nhắc nhở vào `email_queue` để Worker xử lý.

6 Kiến trúc Triển khai (Deployment Architecture)

Mô hình triển khai dựa trên Containerization để đảm bảo tính nhất quán trên mọi môi trường (Dev, Staging, Prod).

6.1 Môi trường đóng gói (Docker)

Cấu trúc `docker-compose.yml` gồm các services:

- `frontend`: Phục vụ các file tĩnh của ứng dụng SPA (Nginx).
- `backend_api`: Container chạy Web Server xử lý API chính.
- `worker`: Container chạy tiến trình background (Consumer / Cronjob).
- `postgres`: Container CSDL PostgreSQL.
- `redis`: Container Redis làm Message Broker và Caching.
- `mailhog` (Chỉ cho Dev): Giả lập SMTP server để test gửi email nội bộ.

Mô hình này giúp mọi thành viên trong team chỉ cần chạy `docker compose up -d` là có đủ môi trường phát triển toàn diện.